



Plan de Pruebas — Casos de Prueba

Documento: TESTING — Divise (Cotizaciones de mercado en tiempo real)
Fecha: 14/08/2026 | Versión: 1.0
Autor: QA / Desarrollo — Divise

1. Información General

PROYECTO	Divise — aplicación web de cotizaciones de divisas y criptomonedas en tiempo real
URL FRONTEND	https://divise-frontend.onrender.com
URL API / BACKEND	https://divise.onrender.com (endpoints bajo /api/*)
BASE DE DATOS	PostgreSQL (Supabase) — tablas: usuarios, divisas, tipos_de_cambio, favoritos, alertas, historial_de_consultas
TECNOLOGÍAS	Frontend: React + Vite + React Router (HashRouter). Backend: Node.js + Express 5 + pg. Autenticación: JWT (24 h) + bcrypt.
APIS EXTERNAS	dolarapi.com (cotizaciones USD/fiat), api.coingecko.com (cripto en ARS), api.bluelytics.com.ar (historial), api.rss2json.com (noticias RSS)
USUARIOS DE PRUEBA	prueba@email.com / 123456 (existente). Cualquier correo nuevo para registro.
ALCANCE	Pruebas funcionales, de seguridad, rendimiento, resiliencia y responsive sobre frontend y backend desplegados.

2. Convenciones de Estado

ESTADO	SIGNIFICADO
APROBADO	Comportamiento verificado, coincide con el resultado esperado.
FALLO DETECTADO	El resultado obtenido difiere del esperado (o la funcionalidad no existe / no responde).
PENDIENTE	Caso diseñado; falta ejecución manual en el entorno indicado.
N/A	No aplica (la funcionalidad no existe o el requisito es irrelevante para la app actual).

3. Datos de prueba sugeridos

DATO	VALOR
Login válido	prueba@email.com / 123456
Login inválido	prueba@email.com / incorrecta
Registro (email nuevo)	Nombre: "Usuario QA", email: qa+[fecha]@divise.com, contraseña: PassTest2024
SQLi / XSS	' OR 1=1 -- ; <script>alert(1)</script>
Monedas	USD (Blue/Oficial), EUR, BRL, BTC, ETH, ARS

4. Casos de prueba provistos (CP-001 a CP-024)

Casos entregados por el cliente, adaptados a la aplicación Divide real (la raíz "/" redirige a /dashboard, que requiere login; las cotizaciones se ven en /dashboard/divisas).

CP-001 Visualización de cotizaciones en la página principal		ALTA	PENDIENTE
Precondiciones	Usuario autenticado en el dashboard (ruta /dashboard o /dashboard/divisas). Servidor operativo. Conexión a Internet.		
Pasos	1. Iniciar sesión e ingresar al dashboard. 2. Ir a la sección "Cotizaciones" (/dashboard/divisas). 3. Esperar la carga completa.		
Resultado esperado	Se muestran las cotizaciones: Oficial, Blue, MEP/Bolsa, CCL (Contado con Liqui), Tarjeta, Mayorista y cripto (BTC/ETH), con compra y venta legibles.		
Resultado obtenido	El endpoint /api/rates responde 200 con datos reales (ej. BTC 93.603.549 / 94.539.584 ARS). La grilla de /dashboard/divisas consume dolarapi + CoinGecko y renderiza las cards. Nota: las cards no muestran Mayorista/Tarjeta/CCL si el proveedor no los lista.		

CP-002 Actualización automática de cotizaciones (sin recargar)		ALTA	FALLO DETECTADO
Precondiciones	Usuario en /dashboard/divisas. Cotizaciones cargadas.		
Pasos	1. Registrar el valor actual de una cotización (ej. Blue). 2. Esperar 60 segundos. 3. Verificar nuevamente el valor.		
Resultado esperado	La información se actualiza automáticamente sin recargar la página.		
Resultado obtenido	NO SE CUMPLE. El frontend consulta las cotizaciones una sola vez al montar la página (useEffect sin interval). Solo cambia el reloj "Actualizado HH:MM:SS". El backend sincroniza cada 5 minutos, pero el frontend no la re-consulta. Requiere recarga manual para ver datos nuevos.		

CP-003 Funcionamiento de la calculadora de conversión		ALTA	APROBADO
Precondiciones	Usuario en /dashboard/calculadora. Cotizaciones cargadas.		
Pasos	1. Ingresar \$100.000 ARS. 2. Seleccionar conversión a USD (Blue/Informal). 3. Observar el resultado.		
Resultado esperado	El resultado coincide con la cotización mostrada, respetando decimales.		
Resultado obtenido	Lógica verificada: resultado = monto ARS / precio venta USD (Informal). Con 100.000 ARS y USD Blue venta 1.540 se obtiene ≈ 64,94 USD. Se aplican entre 2 y 4 decimales.		

CP-004 Validación de ingreso de letras en la calculadora		MEDIA	FALLO DETECTADO
Precondiciones	Usuario en /dashboard/calculadora.		

Pasos	1. Ingresar "abc" en el campo de monto. 2. Observar el resultado.
Resultado esperado	El sistema rechaza el valor y muestra un mensaje de error.
Resultado obtenido	NO SE CUMPLE. El código usa parseFloat(amount) 0, por lo que "abc" se convierte en 0 y el resultado muestra "0,00 USD" sin ningún mensaje de error. Falta validación de entrada.

CP-005	Conversión con valor cero	BAJA	APROBADO
Precondiciones	Usuario en /dashboard/calculadora.		
Pasos	1. Ingresar 0 en el campo de monto. 2. Observar el resultado.		
Resultado esperado	El resultado es 0.		
Resultado obtenido	Muestra "0,00 {moneda}". Correcto.		

CP-006	Persistencia del modo oscuro	MEDIA	N/A
Precondiciones	Usuario en el dashboard.		
Pasos	1. Activar el modo oscuro. 2. Recargar la página (F5).		
Resultado esperado	El modo oscuro permanece activo.		
Resultado obtenido	N/A. La aplicación es 100% oscura y no existe un toggle para alternar tema (en Perfil solo está la opción "Oscuro (Divise Pro)", deshabilitada la clara). No hay nada que persistir.		

CP-007	Visualización correcta en dispositivos móviles	ALTA	PENDIENTE
Precondiciones	Acceso desde un teléfono móvil (o DevTools emulación ≤ 430 px).		
Pasos	1. Abrir la app desde el celular. 2. Navegar por login, dashboard y las distintas secciones.		
Resultado esperado	No existe desplazamiento horizontal; todos los elementos son visibles y utilizables.		
Resultado obtenido	El CSS incluye media queries (auth-wrapper colapsa, menú hamburguesa, grillas responsive). Verificar en dispositivo real con las tablas de Favoritos/Historial (riesgo de overflow).		

CP-008	Comportamiento ante caída de una API externa	CRÍTICA	APROBADO
Precondiciones	Usuario en el dashboard.		
Pasos	1. Simular indisponibilidad de dolarapi/CoinGecko (desconectar red de esas APIs o bloquear dominio). 2. Cargar cotizaciones.		

Resultado esperado	El sistema muestra los últimos datos almacenados (o valores de respaldo) y no se cae.
Resultado obtenido	fetchRates() captura el error y devuelve un arreglo de respaldo hardcodeado (Blue 1540, Oficial 1515, etc.). La app continúa funcionando sin crashear.

CP-009 Carga inicial bajo conexión lenta

MEDIA

APROBADO

Precondiciones	Red 3G lenta emulada en DevTools.
Pasos	1. Abrir /dashboard/divisas. 2. Observar el proceso de carga.
Resultado esperado	Skeletons o indicadores de carga; la interfaz no aparece vacía.
Resultado obtenido	/dashboard/divisas muestra 6 cards skeleton mientras carga. El inicio muestra "Cargando cotizaciones en vivo...". Cumple.

CP-010 Protección contra SQL Injection (login)

CRÍTICA

APROBADO

Precondiciones	Usuario en la página de login.
Pasos	1. Ingresar ' OR 1=1 -- en el campo de email. 2. Ingresar cualquier contraseña. 3. Presionar "Iniciar sesión".
Resultado esperado	La entrada es rechazada; no se altera la base de datos.
Resultado obtenido	authService usa consultas parametrizadas (\$1) en userModel; el backend responde 401 "Credenciales inválidas". No hay riesgo de inyección en este flujo.

CP-011 Verificación del tiempo de respuesta de la API

ALTA

APROBADO

Precondiciones	Backend operativo. curl/Postman disponible.
Pasos	1. GET a https://divise.onrender.com/api/rates (endpoint real de cotizaciones). 2. Medir el tiempo de respuesta.
Resultado esperado	Respuesta en menos de 500 ms.
Resultado obtenido	GET /api/rates → 200 en ~453 ms (medición: 0.453654 s). Cumple (borde superior; puede variar con el cold start de Render).

CP-012 Navegación completa del usuario

ALTA

PENDIENTE

Precondiciones	Sistema funcionando. Usuario autenticado.
-----------------------	---

Pasos	1. Ingresar al sitio. 2. Consultar cotizaciones. 3. Usar la calculadora. 4. Consultar gráficos y noticias. 5. Recargar la página.
Resultado esperado	Todas las funcionalidades operan sin errores.
Resultado obtenido	Flujo de navegación HashRouter verificado (login → dashboard → secciones). Los botones sin acción de la calculadora/perfil no producen errores pero tampoco respuestas (ver CP-041, CP-042, CP-071).

CP-013	Inicio de sesión correcto	CRÍTICA	APROBADO
Precondiciones	Usuario registrado. Credenciales válidas (prueba@email.com / 123456).		
Pasos	1. Ingresar al formulario de login. 2. Escribir email y contraseña correctos. 3. Presionar "Iniciar sesión".		
Resultado esperado	Acceso correcto; redirige al dashboard.		
Resultado obtenido	POST /api/auth/login → 200 con { user, token }. El frontend guarda el token en localStorage y redirige a /dashboard. Verificado.		

CP-014	Inicio de sesión con contraseña incorrecta	CRÍTICA	APROBADO
Precondiciones	Usuario registrado (prueba@email.com).		
Pasos	1. Ingresar email correcto. 2. Ingresar contraseña incorrecta. 3. Presionar "Iniciar sesión".		
Resultado esperado	Acceso rechazado; mensaje de credenciales incorrectas.		
Resultado obtenido	POST /api/auth/login → 401 {"error":"Credenciales inválidas"}; el frontend muestra el mensaje en el formulario. Verificado.		

CP-015	Inicio de sesión con correo inexistente	ALTA	APROBADO
Precondiciones	Email no registrado.		
Pasos	1. Ingresar inexistente@ejemplo.com. 2. Ingresar una contraseña cualquiera. 3. Presionar "Iniciar sesión".		
Resultado esperado	No permite el acceso; muestra mensaje de error.		
Resultado obtenido	Backend responde 401 "Credenciales inválidas" (no distingue email inexistente de contraseña mala, lo cual es correcto por seguridad). Verificado.		

CP-016 Campos vacíos en inicio de sesión

ALTA

APROBADO

Precondiciones	Formulario de login disponible.
Pasos	1. Dejar email vacío. 2. Dejar contraseña vacía. 3. Presionar "Iniciar sesión".
Resultado esperado	Mensajes de validación; no se envía la solicitud.
Resultado obtenido	Los inputs tienen el atributo required → el navegador bloquea el envío mostrando el tooltip nativo. No se dispara la request. Verificado.

CP-017 Cierre de sesión

ALTA

APROBADO

Precondiciones	Usuario autenticado en el dashboard.
Pasos	1. Presionar el botón "Cerrar sesión" (icono de logout en la barra superior).
Resultado esperado	La sesión finaliza; redirige a /login.
Resultado obtenido	logout() elimina divide_token y divide_user de localStorage y redirige a /login. Verificado.

CP-018 Recuperación de contraseña con correo válido

CRÍTICA

FALLO DETECTADO

Precondiciones	Usuario registrado. Página de login.
Pasos	1. Presionar "¿Olvidaste tu contraseña?". 2. Ingresar el correo. 3. Solicitar recuperación.
Resultado esperado	Se envía un correo de recuperación y se informa al usuario.
Resultado obtenido	FALLO DE UI. El enlace apunta a /forgot, ruta que NO existe en el frontend (App.jsx no la define) → el router cae en el catch-all y redirige a /login. El backend /api/auth/forgot-password existe y funciona (genera token de 30 min e intenta envío de email), pero no es accesible desde la interfaz.

CP-019 Recuperación de contraseña con correo inexistente

ALTA

PENDIENTE

Precondiciones	Correo no registrado.
Pasos	1. Acceder a la recuperación (a nivel API: POST /api/auth/forgot-password). 2. Ingresar correo-inexistente@ejemplo.com.
Resultado esperado	Mensaje informativo genérico; no se revela si el correo existe.
Resultado obtenido	El backend devuelve {"success":true,"message":"Si el correo existe, enviaremos un enlace."} sin filtrar existencia. Correcto por seguridad (eludir revelar emails). No accesible desde la UI por la falla del CP-018.

CP-020 Cambio de contraseña mediante enlace de recuperación**CRÍTICA****PENDIENTE**

Precondiciones	Token de recuperación válido (emitido por forgot-password).
Pasos	1. POST /api/auth/reset-password con { token, newPassword }. 2. Iniciar sesión con la nueva contraseña.
Resultado esperado	Contraseña actualizada; login con la nueva contraseña funciona.
Resultado obtenido	El endpoint reset-password valida el token, hashlea con bcrypt y actualiza password_hash. Requiere ejecución vía API/Postman (sin UI de reset). Pendiente de ejecución end-to-end con email real.

CP-021 Enlace de recuperación expirado**ALTA****PENDIENTE**

Precondiciones	Token vencido (expiración 30 min en reset_token_expires).
Pasos	1. Usar el token vencido en reset-password.
Resultado esperado	El sistema informa que el enlace expiró y solicita una nueva recuperación.
Resultado obtenido	getUserByResetToken descarta tokens cuya expiración ya pasó → devuelve 400 "Token inválido o expirado". Correcto a nivel API.

CP-022 Registro de usuario nuevo**CRÍTICA****APROBADO**

Precondiciones	Usuario no registrado. Página /register.
Pasos	1. Completar nombre, email y contraseña (mín. 8 caracteres) y confirmación. 2. Aceptar términos. 3. Confirmar registro.
Resultado esperado	Cuenta creada; usuario guardado en la base; ingresa al dashboard.
Resultado obtenido	POST /api/auth/register → 201 con { user, token }. El usuario se inserta en usuarios y se redirige a /dashboard. Verificado (testreg@test.com creado durante la sesión de prueba).

CP-023 Registro con correo ya existente**ALTA****APROBADO**

Precondiciones	Existe una cuenta con ese correo (ej. prueba@email.com).
Pasos	1. Registrar con el correo existente.
Resultado esperado	Rechazo; mensaje de correo en uso.
Resultado obtenido	Backend valida existencia → 400 "El email ya está registrado"; el frontend muestra el mensaje. Verificado.

CP-024 Validación de contraseña mínima en registro**ALTA****APROBADO**

Precondiciones	Formulario de registro disponible.
Pasos	1. Ingresar contraseña "12" (menor a 8). 2. Intentar registrarse.
Resultado esperado	Mensaje de validación; no se crea la cuenta.
Resultado obtenido	El frontend valida longitud ≥ 8 y coincidencia de confirmación antes de llamar a la API. Observación: el backend NO valida longitud mínima (aceptaría "12" vía API directa) — recomendación endurecer en servidor.

5. Cotizaciones (Inicio y Divisas)

Comportamiento de las pantallas /dashboard (Inicio) y /dashboard/divisas.

CP-025 KPIs del inicio muestran valores reales		ALTA	APROBADO
Precondiciones	Usuario autenticado en /dashboard.		
Pasos	1. Ingresar al dashboard. 2. Verificar las 4 tarjetas (Blue, Oficial, Euro, Bitcoin).		
Resultado esperado	Precios reales (fallback 1540/1515/1722/64200 si falla el proveedor).		
Resultado obtenido	Usa fetchRates() y valores reales de dolarapi/CoinGecko; si falla, usa defaults. Correcto.		

CP-026 Filtro por categoría en /dashboard/divisas		MEDIA	APROBADO
Precondiciones	Usuario en /dashboard/divisas.		
Pasos	1. Seleccionar "Divisas", "Cripto" y "Todos".		
Resultado esperado	La grilla filtra según categoría (cripto = BTC/ETH por tipo_mercado 'Cripto' o códigos USDT/USDC).		
Resultado obtenido	Filtrado por isCrypto(); funciona en estado local. Correcto.		

CP-027 Búsqueda de divisas por nombre o código		MEDIA	APROBADO
Precondiciones	Usuario en /dashboard/divisas.		
Pasos	1. Escribir "btc" o "bitcoin" en el buscador.		
Resultado esperado	Se filtran las cards que coinciden (case-insensitive).		
Resultado obtenido	Filtra por nombre y código con toLowerCase. Correcto.		

CP-028 Búsqueda sin resultados		BAJA	APROBADO
Precondiciones	Usuario en /dashboard/divisas.		
Pasos	1. Buscar "zzzzz".		
Resultado esperado	Mensaje "No se encontraron cotizaciones con esos filtros."		
Resultado obtenido	Muestra el estado vacío con icono. Correcto.		

CP-029 Click en una card de divisa navega al gráfico		ALTA	APROBADO
Precondiciones	Usuario en /dashboard/divisas o inicio.		

Pasos	1. Clic en una card (ej. Dólar Blue).
Resultado esperado	Navega a /dashboard/graficos?moneda=USD&mercado=Informal.
Resultado obtenido	Navegación con query params implementada (click y Enter/espacio). Correcto.

CP-030 Botón de favorito en la card no dispara navegación

MEDIA

FALLO DETECTADO

Precondiciones	Usuario en /dashboard/divisas.
Pasos	1. Clic en la estrella de una card.
Resultado esperado	Agrega/quita favorito (persistente) sin navegar al gráfico.
Resultado obtenido	NO SE CUMPLE. La estrella solo hace stopPropagation (evita navegar) pero no llama a /api/favorites/toggle ni actualiza ningún estado: no guarda nada. No hay persistencia.

CP-031 Reloj de última actualización

BAJA

APROBADO

Precondiciones	Usuario en /dashboard/divisas.
Pasos	1. Observar el indicador "Actualizado HH:MM:SS".
Resultado esperado	El reloj se actualiza cada segundo.
Resultado obtenido	setInterval(1s) sobre el estado `now`. Correcto.

CP-032 Acceso directo a /dashboard/divisas sin sesión

CRÍTICA

APROBADO

Precondiciones	Sesión no iniciada.
Pasos	1. Abrir la URL con #/dashboard/divisas.
Resultado esperado	Redirige a /login (ruta protegida).
Resultado obtenido	PrivateRoute redirige a /login si no hay token. Verificado.

6. Gráficos

Pantalla /dashboard/graficos.

CP-033 Gráfico histórico de USD (Blue/Oficial)		ALTA	APROBADO
Precondiciones	Usuario autenticado. Internet disponible.		
Pasos	1. Abrir /dashboard/graficos?moneda=USD&mercado=Informal.		
Resultado esperado	Gráfico de 30 días con datos reales de bluelytics.		
Resultado obtenido	fetchUsdHistory('Blue') → api.bluelytics.com.ar. Correcto (con datos reales).		

CP-034 Gráfico histórico de cripto (BTC/ETH) en ARS		ALTA	PENDIENTE
Precondiciones	Usuario autenticado. CoinGecko disponible.		
Pasos	1. Abrir /dashboard/graficos?moneda=BTC.		
Resultado esperado	Gráfico de BTC/ETH con valores en ARS (market_chart).		
Resultado obtenido	fetchCryptoHistory('bitcoin' 'ethereum'). Se recomienda validar formato y manejo de límites de la API gratuita.		

CP-035 Parámetros inválidos o vacíos en la URL de gráficos		MEDIA	PENDIENTE
Precondiciones	Usuario autenticado.		
Pasos	1. Abrir /dashboard/graficos sin query params o con moneda inexistente (ej. XXXX).		
Resultado esperado	Estado vacío o mensaje claro, sin crasheos.		
Resultado obtenido	A evaluar (la página depende de los params; debe validar ausencia de datos).		

CP-036 Fallback cuando la API de historial cae		MEDIA	PENDIENTE
Precondiciones	bluelytics/CoinGecko no disponible.		
Pasos	1. Abrir la pantalla de gráficos.		
Resultado esperado	Se muestra mensaje de error/estado vacío; la app no se cae.		
Resultado obtenido	Pendiente de validar el manejo de error en Graficos.jsx.		

7. Calculadora

Pantalla /dashboard/calculadora.

CP-037 Conversión USD → ARS		ALTA	APROBADO
Precondiciones	Usuario en la calculadora. Cotizaciones cargadas.		
Pasos	1. Desde USD, monto 100. 2. Hacia ARS.		
Resultado esperado	100 × precio venta USD (Blue) = resultado en ARS.		
Resultado obtenido	getARSValue('USD') usa el mercado 'Informal' (venta) → 100 × 1540 = 154.000,00 ARS. Correcto.		

CP-038 Botón de intercambio (swap) de monedas		BAJA	APROBADO
Precondiciones	Usuario en la calculadora.		
Pasos	1. Presionar el botón ⇄.		
Resultado esperado	Intercambia las monedas "Desde" y "Hacia".		
Resultado obtenido	handleSwap intercambia fromCurrency/toCurrency. Correcto.		

CP-039 Monto vacío en la calculadora		MEDIA	FALLO DETECTADO
Precondiciones	Usuario en la calculadora.		
Pasos	1. Borrar el monto.		
Resultado esperado	Se muestra 0 o se solicita el monto.		
Resultado obtenido	parseFloat("") 0 → muestra 0,00. No valida ni muestra mensaje; comportamiento aceptable pero sin feedback.		

CP-040 Monto negativo		MEDIA	FALLO DETECTADO
Precondiciones	Usuario en la calculadora.		
Pasos	1. Ingresar -500.		
Resultado esperado	El sistema rechaza valores negativos.		
Resultado obtenido	Se calcula un resultado negativo (no hay validación de signo). Fallo de validación menor.		

CP-041 Monto decimal con coma/punto		BAJA	APROBADO
Precondiciones	Usuario en la calculadora.		

Pasos	1. Ingresar "1.000,50".
Resultado esperado	Se interpreta como 1000.50.
Resultado obtenido	reemplaza ',' por '.' → parseFloat("1000.50"). Correcto.

CP-042 Conversión con moneda sin cotización (JPY, GBP, CAD, CHF, AUD)

MEDIA

FALLO DETECTADO

Precondiciones	Usuario en la calculadora.
Pasos	1. Seleccionar JPY como origen o destino e ingresar un monto.
Resultado esperado	Conversión válida o mensaje de "moneda no disponible".
Resultado obtenido	Las opciones incluyen JPY/GBP/CAD/CHF/AUD, pero dolarapi solo devuelve USD/EUR/BRL (y similares). getARSValue devuelve 0 → resultado 0,00 sin explicación. Fallo de UX.

CP-043 Botón "Convertir" ejecuta la conversión

ALTA

FALLO DETECTADO

Precondiciones	Usuario en la calculadora.
Pasos	1. Ingresar monto y presionar "Convertir".
Resultado esperado	La conversión se ejecuta y registra en el historial.
Resultado obtenido	NO SE CUMPLE. El botón "Convertir" no tiene onClick: no hace nada. El resultado solo se calcula en vivo conforme se escribe. El historial de conversiones es un mock estático.

CP-044 Botón "Agregar a favoritos"

MEDIA

FALLO DETECTADO

Precondiciones	Usuario en la calculadora.
Pasos	1. Presionar "Agregar a favoritos".
Resultado esperado	Agrega el par a favoritos con confirmación.
Resultado obtenido	NO SE CUMPLE. Botón sin onClick. No hay acción.

CP-045 Precisión de decimales en el resultado

BAJA

APROBADO

Precondiciones	Usuario en la calculadora.
Pasos	1. Ejecutar conversiones con resultados fraccionarios.
Resultado esperado	Mínimo 2 y máximo 4 decimales.
Resultado obtenido	toLocaleString('es-AR', {min:2, max:4}). Correcto.

8. Noticias

Pantalla /dashboard/noticias.

CP-046 Carga de noticias desde RSS		MEDIA	PENDIENTE
Precondiciones	Usuario autenticado. api.rss2json.com accesible.		
Pasos	1. Abrir /dashboard/noticias.		
Resultado esperado	Se muestran hasta 12 noticias de Ámbito (economía) con imagen, título y fecha.		
Resultado obtenido	Depende de rss2json (sin api_key, el servicio público es inestable). Verificar conectividad real; existe estado de carga "Conectando...".		

CP-047 Apertura de noticia en nueva pestaña		MEDIA	PENDIENTE
Precondiciones	Noticias cargadas.		
Pasos	1. Clic en una tarjeta de noticia.		
Resultado esperado	Abre el artículo en una pestaña nueva (target=_blank, rel=noreferrer).		
Resultado obtenido	Implementado con a target="_blank" rel="noreferrer". Correcto.		

CP-048 Fallo del proveedor de noticias		MEDIA	PENDIENTE
Precondiciones	rss2json no disponible.		
Pasos	1. Abrir /dashboard/noticias.		
Resultado esperado	Mensaje de error claro; la app no se cae.		
Resultado obtenido	catch → setError("Error al conectar con el servidor de noticias."). Pendiente de validar en ejecución.		

9. Alertas

Pantalla /dashboard/alertas. IMPORTANTE: el frontend usa datos mock locales y NO consume /api/alerts.

CP-049 Crear una alerta nueva		ALTA	APROBADO
Precondiciones	Usuario en /dashboard/alertas.		
Pasos	1. Presionar "Nueva Alerta". 2. Completar divisa, condición y valor. 3. Guardar.		
Resultado esperado	La alerta aparece en la grilla como activa.		
Resultado obtenido	handleSaveAlert agrega al estado local y cierra el modal. Funciona en la sesión actual.		

CP-050 Eliminar una alerta		MEDIA	APROBADO
Precondiciones	Existe al menos una alerta.		
Pasos	1. Presionar "Eliminar" en una alerta.		
Resultado esperado	La alerta se elimina de la lista.		
Resultado obtenido	handleDelete filtra por id. Funciona en sesión.		

CP-051 Estado vacío de alertas		BAJA	PENDIENTE
Precondiciones	Lista vacía (eliminar todas o estado inicial sin datos).		
Pasos	1. Observar la pantalla con 0 alertas.		
Resultado esperado	Mensaje "No tenés alertas activas" con botón para crear.		
Resultado obtenido	Render condicional implementado. Correcto.		

CP-052 Persistencia de alertas tras recargar		CRÍTICA	FALLO DETECTADO
Precondiciones	Se creó al menos una alerta.		
Pasos	1. Recargar la página.		
Resultado esperado	Las alertas persisten (están en la base).		
Resultado obtenido	NO SE CUMPLE. Las alertas viven en useState local (mock). Al recargar, vuelven las 2 alertas de ejemplo hardcodedas. No se guardan en /api/alerts ni en la base de datos.		

CP-053 Integración con el backend /api/alerts		CRÍTICA	FALLO DETECTADO
Precondiciones	Token válido.		

Pasos	1. Verificar si Alertas.jsx realiza GET/POST/DELETE a /api/alerts.
Resultado esperado	Las alertas se leen/crean/eliminan vía API autenticada.
Resultado obtenido	NO SE CUMPLE. El backend está implementado (GET/POST/DELETE con verifyToken + alertService), pero el frontend no lo usa. La funcionalidad de alertas de la UI es completamente mock.

CP-054 Validación del modal de alerta

MEDIA

PENDIENTE

Precondiciones	Modal de alerta abierto.
Pasos	1. Guardar con campos vacíos o valor no numérico.
Resultado esperado	Validación con mensajes claros.
Resultado obtenido	Pendiente de revisar AlertModal.jsx (no consume backend; validar comportamiento).

10. Favoritos

Pantalla /dashboard/favoritos. Frontend con datos mock; el backend /api/favorites existe pero no se consume.

CP-055 Agregar / quitar favorito		MEDIA	APROBADO
Precondiciones	Usuario en /dashboard/favoritos.		
Pasos	1. Clic en la estrella de una fila.		
Resultado esperado	Cambia el estado favorita/no favorita y muestra toast.		
Resultado obtenido	toggleFavorite + toast "Agregado/Quitado de favoritas". Funciona en sesión.		

CP-056 Búsqueda de divisas en favoritos		BAJA	APROBADO
Precondiciones	Usuario en /dashboard/favoritos.		
Pasos	1. Buscar por nombre o código.		
Resultado esperado	Filtra la tabla.		
Resultado obtenido	Filtrado case-insensitive + reset de página. Correcto.		

CP-057 Ordenamiento de favoritos		BAJA	APROBADO
Precondiciones	Usuario en /dashboard/favoritos.		
Pasos	1. Cambiar orden: favoritas primero, código, precio mayor/menor.		
Resultado esperado	La tabla reordena correctamente.		
Resultado obtenido	Ordenamiento implementado por sortOrder. Correcto.		

CP-058 Paginación en favoritos		BAJA	APROBADO
Precondiciones	Más de 5 favoritos.		
Pasos	1. Usar "Anterior/Siguiente" y los números de página.		
Resultado esperado	Paginación de 5 ítems, botones deshabilitados en los extremos.		
Resultado obtenido	Implementado (itemsPerPage=5, totalPages, disable en bordes). Correcto.		

CP-059 Quick Convert abre la calculadora precargada		MEDIA	APROBADO
Precondiciones	Usuario en /dashboard/favoritos.		
Pasos	1. Presionar "Quick Convert" en una fila.		
Resultado esperado	Navega a la calculadora con la moneda precargada.		

Resultado obtenido

navigate('/dashboard/calculadora', { state: { currency } }) y Calculadora lee location.state. Correcto.

CP-060 Botón "Re-consultar"

BAJA

PENDIENTE

Precondiciones	Usuario en /dashboard/favoritos.
Pasos	1. Presionar "Re-consultar".
Resultado esperado	Actualiza la cotización de la divisa y muestra el nuevo valor.
Resultado obtenido	Solo muestra un toast con el precio mock (no re-consulta la API). Se considera fallo menor de funcionalidad real.

CP-061 Persistencia de favoritos tras recargar

CRÍTICA

FALLO DETECTADO

Precondiciones	Se marcaron/quitaron favoritos.
Pasos	1. Recargar la página.
Resultado esperado	Los favoritos se mantienen (guardados en DB).
Resultado obtenido	NO SE CUMPLE. Lista mock hardcodeda en useState; los cambios se pierden al recargar. El backend /api/favoritos (toggle autenticado) no se usa.

11. Historial de Consultas

Pantalla /dashboard/historial. Frontend con datos mock; no usa historial_de_consultas de la DB.

CP-062 Filtro por rango de fechas		MEDIA	FALLO DETECTADO
Precondiciones	Usuario en /dashboard/historial.		
Pasos	1. Elegir rango de fechas y aplicar filtros.		
Resultado esperado	El historial se filtra por el rango.		
Resultado obtenido	NO SE CUMPLE. Los filtros solo muestran un toast "Filtros de historial aplicados"; la lista (mock) no se filtra por fecha.		

CP-063 Búsqueda en historial		BAJA	APROBADO
Precondiciones	Usuario en /dashboard/historial.		
Pasos	1. Buscar por nombre/código.		
Resultado esperado	Filtra los registros.		
Resultado obtenido	Filtrado por nombre/código + reset página. Correcto.		

CP-064 Ordenamiento recientes/antiguas		BAJA	APROBADO
Precondiciones	Usuario en /dashboard/historial.		
Pasos	1. Cambiar orden.		
Resultado esperado	Ordena por id (recientes = mayor id).		
Resultado obtenido	Correcto sobre los datos mock.		

CP-065 Paginación del historial		BAJA	APROBADO
Precondiciones	Más de 6 registros.		
Pasos	1. Navegar páginas.		
Resultado esperado	Paginación de 6 ítems funcional.		
Resultado obtenido	Correcto sobre datos mock.		

CP-066 Limpiar filtros		BAJA	APROBADO
Precondiciones	Usuario en /dashboard/historial.		
Pasos	1. Presionar "Limpiar filtros".		

Resultado esperado	Se vacían fechas y búsqueda, vuelve a página 1, toast.
Resultado obtenido	Correcto.

CP-067 Persistencia y fuentes reales del historial		ALTA	FALLO DETECTADO
Precondiciones	Usuario autenticado con consultas previas.		
Pasos	1. Consultar una cotización y luego abrir el historial.		
Resultado esperado	La consulta queda registrada en historial_de_consultas.		
Resultado obtenido	NO SE CUMPLE. El historial es un mock estático (fechas de 2024); no registra consultas reales ni lee la tabla historial_de_consultas.		

12. Perfil

Pantalla /dashboard/perfil.

CP-068 Visualización de datos del usuario		MEDIA	APROBADO
Precondiciones	Usuario autenticado.		
Pasos	1. Abrir /dashboard/perfil.		
Resultado esperado	Muestra nombre, email y avatar con inicial.		
Resultado obtenido	Lee user de AuthContext (localStorage). Correcto.		

CP-069 Editar perfil		MEDIA	FALLO DETECTADO
Precondiciones	Usuario autenticado.		
Pasos	1. Presionar "Editar Perfil".		
Resultado esperado	Permite editar nombre/email y guardar.		
Resultado obtenido	NO SE CUMPLE. Botón sin onClick. No hay edición.		

CP-070 Cambio de tema desde el perfil		MEDIA	N/A
Precondiciones	Usuario autenticado.		
Pasos	1. Cambiar el tema en Preferencias Visuales.		
Resultado esperado	El tema cambia y persiste.		
Resultado obtenido	N/A. Solo existe la opción "Oscuro (Divise Pro)"; "Claro" está deshabilitada. El select no aplica ningún cambio.		

CP-071 Cambiar contraseña desde el perfil		MEDIA	FALLO DETECTADO
Precondiciones	Usuario autenticado.		
Pasos	1. Presionar "Cambiar" en Seguridad.		
Resultado esperado	Abre flujo de cambio de contraseña.		
Resultado obtenido	NO SE CUMPLE. Botón sin onClick.		

CP-072 Activar 2FA		BAJA	FALLO DETECTADO
Precondiciones	Usuario autenticado.		
Pasos	1. Presionar "Activar" en 2FA.		

Resultado esperado	Inicia el flujo de configuración 2FA.
Resultado obtenido	NO SE CUMPLE. Botón sin onClick (placeholder visual).

13. Autenticación y Seguridad

Validación de sesión, tokens y vectores de ataque.

CP-073 Sesión persistente tras recargar (localStorage)		ALTA	APROBADO
Precondiciones	Usuario autenticado.		
Pasos	1. F5 en el dashboard.		
Resultado esperado	Permanece en el dashboard sin re-login.		
Resultado obtenido	AuthContext inicializa desde divide_token/divide_user. Correcto.		

CP-074 Token JWT expirado (24 h)		ALTA	PENDIENTE
Precondiciones	Token con exp pasado o firmado con expiresIn corto.		
Pasos	1. Llamar a un endpoint protegido con ese token.		
Resultado esperado	Respuesta 401 "Token inválido o expirado".		
Resultado obtenido	verifyToken usa jwt.verify; un token expirado devuelve 401. Correcto a nivel API.		

CP-075 Token manipulado / no firmado		CRÍTICA	APROBADO
Precondiciones	Token alterado (payload modificado) o inventado.		
Pasos	1. Enviar "Bearer abc.def.ghi" a /api/users/me.		
Resultado esperado	401; no se concede acceso.		
Resultado obtenido	jwt.verify falla la firma → 401. Correcto.		

CP-076 Acceso a rutas de API sin token		CRÍTICA	APROBADO
Precondiciones	Sin Authorization header.		
Pasos	1. GET /api/alerts, /api/favorites, /api/users/me sin token.		
Resultado esperado	401.		
Resultado obtenido	router.use(verifyToken) en esas rutas → 401 "No se proporcionó token". Correcto.		

CP-077 SQL Injection en búsquedas y parámetros		CRÍTICA	APROBADO
Precondiciones	Acceso a endpoints con parámetros (rates, alerts, favorites).		
Pasos	1. Enviar payloads tipo ' OR 1=1 -- en campos/parámetros.		
Resultado esperado	Sin efectos; sin inyección.		

Resultado obtenido	Todas las consultas usan parámetros \$n (pg) o valores fijos. Correcto.
--------------------	---

CP-078 XSS en campos de texto (nombre, búsquedas)

ALTA

APROBADO

Precondiciones	Formularios de registro/login y buscadores.
Pasos	1. Ingresar <code><script>alert(1)</script></code> en nombre/búsqueda.
Resultado esperado	Se muestra como texto; no se ejecuta.
Resultado obtenido	React escapa el contenido por defecto en los componentes revisados. NOTA: Noticias.jsx usa dangerouslySetInnerHTML en el excerpt (contenido RSS externo) — riesgo a evaluar.

CP-079 Contraseñas almacenadas con hash (bcrypt)

CRÍTICA

APROBADO

Precondiciones	Acceso de lectura a la tabla usuarios.
Pasos	1. Verificar la columna password_hash de un usuario.
Resultado esperado	No hay contraseñas en texto plano.
Resultado obtenido	bcrypt.hash(password, 10) en registro y reset. Columnas password_hash con prefijo \$2b\$. Correcto.

CP-080 Login con email en mayúsculas o con espacios

MEDIA

FALLO DETECTADO

Precondiciones	Usuario prueba@email.com.
Pasos	1. Ingresar "PRUEBA@EMAIL.COM" o " prueba@email.com ".
Resultado esperado	Se normaliza (lowercase/trim) y autentica.
Resultado obtenido	La búsqueda de email es exacta sin trim/lowercase → "PRUEBA@EMAIL.COM" no matchea y devuelve 401. Fallo menor de UX (recomendable normalizar).

CP-081 Enlace "¿Olvidaste tu contraseña?" no redirige a una página real

ALTA

FALLO DETECTADO

Precondiciones	Página de login.
Pasos	1. Clic en el enlace de recuperación.
Resultado esperado	Acceso a un formulario de recuperación.
Resultado obtenido	El enlace apunta a /forgot (no definida) → catch-all → redirige a /dashboard → PrivateRoute → /login. El usuario queda donde empezó.

CP-082 Rate limiting / protección de fuerza bruta

ALTA

FALLO DETECTADO

Precondiciones	Backend accesible.
----------------	--------------------

Pasos	1. Enviar N intentos de login consecutivos fallidos.
Resultado esperado	Limitación de intentos o delay tras varios fallos.
Resultado obtenido	No existe rate limiting ni bloqueo por intentos fallidos (se recomienda agregar express-rate-limit). Riesgo de fuerza bruta.

14. API / Backend

Endpoints REST de divise.onrender.com.

CP-083 GET /api/rates

ALTA

APROBADO

Precondiciones	Backend operativo, DB conectada.
Pasos	1. GET /api/rates.
Resultado esperado	200 con arreglo de cotizaciones {codigo, nombre, tipo, mercado, compra, venta, fecha}.
Resultado obtenido	200 en ~0.45s con datos reales (BTC, USD, EUR...). Verificado.

CP-084 POST /api/auth/register y /login

CRÍTICA

APROBADO

Precondiciones	Backend operativo.
Pasos	1. POST /api/auth/register con datos válidos (→201). 2. POST /api/auth/login (→200).
Resultado esperado	Códigos 201/200 con token JWT.
Resultado obtenido	Verificado durante la sesión (register 201, login 200). Correcto.

CP-085 POST /api/alerts y DELETE con autenticación

ALTA

PENDIENTE

Precondiciones	Token válido de un usuario.
Pasos	1. POST /api/alerts {codigo_divisa, condicion, valor_limite} → 201. 2. DELETE /api/alerts/:id → 200. 3. DELETE de una alerta de otro usuario → 404.
Resultado esperado	Crear, eliminar y no poder eliminar alertas ajenas.
Resultado obtenido	Lógica implementada en alertRoutes + alertService. Pendiente de ejecución end-to-end.

CP-086 POST /api/favorites/toggle

MEDIA

PENDIENTE

Precondiciones	Token válido.
Pasos	1. POST /api/favorites/toggle {codigo_divisa:'USD'} → {isFavorite:true}. 2. Repetir → {isFavorite:false}. 3. POST con divisa inexistente → 404.
Resultado esperado	Toggle correcto y 404 para divisa inválida.
Resultado obtenido	Lógica implementada. Pendiente de ejecución.

CP-087 GET /api/users/me y listado de usuarios

MEDIA

FALLO DETECTADO

Precondiciones	Token válido.
Pasos	1. GET /api/users/me → perfil propio. 2. GET /api/users → lista de todos los usuarios.
Resultado esperado	El listado no debería exponer datos sensibles a cualquier usuario autenticado.
Resultado obtenido	GET /api/users lista todos los usuarios para cualquier sesión (sin rol). Riesgo de exposición de datos (emails). Recomendación: restringir por rol.

CP-088 Endpoints de salud /healthz y raíz /api

MEDIA

APROBADO

Precondiciones	Backend operativo.
Pasos	1. GET /healthz → 200. 2. GET /api → 200 con lista de endpoints.
Resultado esperado	200 en ambos.
Resultado obtenido	Verificado (200). Correcto.

CP-089 Sincronización automática de cotizaciones (backend)

ALTA

APROBADO

Precondiciones	Backend desplegado.
Pasos	1. Observar logs al iniciar y cada 5 minutos.
Resultado esperado	"Cotizaciones sincronizadas con éxito" al start y cada 5 min; alertas verificadas.
Resultado obtenido	Verificado en ejecución local: sync OK al arrancar con datos reales en tipos_de_cambio. Correcto.

CP-090 Validación de body en endpoints (campos requeridos)

MEDIA

APROBADO

Precondiciones	Backend operativo.
Pasos	1. POST /api/auth/login sin password → 400. 2. POST /api/alerts sin campos → 400.
Resultado esperado	400 con mensaje.
Resultado obtenido	authService lanza 400 "Email y contraseña obligatorios"; alertRoutes valida campos → 400. Correcto.

CP-091 CORS

MEDIA

PENDIENTE

Precondiciones	Backend con cors() habilitado.
Pasos	1. Realizar request cross-origin desde un origen arbitrario.
Resultado esperado	cors() abierto permite cualquier origen (aceptable para API pública; documentar).

Resultado obtenido

app.use(cors()) sin restricciones. Correcto para el modelo actual.

15. Resiliencia y Rendimiento

CP-092 Sin conexión a Internet		CRÍTICA	PENDIENTE
Precondiciones	Red desconectada.		
Pasos	1. Intentar login y cargar dashboard.		
Resultado esperado	Mensajes de error claros ("Servidor backend no disponible"), sin crasheos.		
Resultado obtenido	request() captura errores y avisa en consola; el login muestra el mensaje del error. El dashboard usaría fallbacks de cotizaciones. Pendiente de validar UX en dispositivo real sin red.		

CP-093 Backend caído (Render dormido / error 5xx)		CRÍTICA	PENDIENTE
Precondiciones	Backend no disponible o devolviendo 500.		
Pasos	1. Intentar iniciar sesión.		
Resultado esperado	El frontend muestra un error comprensible y no queda colgado.		
Resultado obtenido	Pendiente de validar manejo de timeout/retry (fetch sin timeout explícito puede demorar hasta el cold start de Render ~30-60s).		

CP-094 Tiempo de carga inicial (bundle)		MEDIA	PENDIENTE
Precondiciones	Red normal.		
Pasos	1. Cargar la app y medir en DevTools (Network/Performance).		
Resultado esperado	Carga razonable; bundle actual ~428 KB (gzip ~139 KB).		
Resultado obtenido	Bundle medido en build (428.04 kB, gzip 139.11 kB). Aceptable; se puede mejorar con code-splitting.		

CP-095 Concurrencia: múltiples usuarios		MEDIA	PENDIENTE
Precondiciones	Varios usuarios simultáneos.		
Pasos	1. Login, consultas y registro en paralelo.		
Resultado esperado	Sin errores de concurrencia (pool de conexiones pg).		
Resultado obtenido	Pendiente de prueba de carga (Render free: 512 MB RAM, puede dormir).		

CP-096 Recarga durante la carga de datos		BAJA	PENDIENTE
Precondiciones	Pantalla cargando cotizaciones/noticias.		
Pasos	1. F5 en medio de la carga.		

Resultado esperado	Sin estados corruptos; se vuelve a cargar normalmente.
Resultado obtenido	Pendiente (los estados son locales; no debería haber corrupción).

16. Responsive y Compatibilidad

CP-097 Vista móvil 360 px sin scroll horizontal

ALTA**PENDIENTE**

Precondiciones	DevTools con viewport 360×800.
Pasos	1. Recorrer login, dashboard y todas las secciones.
Resultado esperado	Sin overflow horizontal; elementos accesibles.
Resultado obtenido	Diseño responsive con media queries; las tablas de Favoritos/Historial son el punto de mayor riesgo de overflow. Validar.

CP-098 Menú de navegación móvil (hamburguesa)

MEDIA**PENDIENTE**

Precondiciones	Viewport móvil.
Pasos	1. Abrir/cerrar el menú y navegar a cada ítem.
Resultado esperado	El menú despliega, navega y cierra al seleccionar.
Resultado obtenido	nav-mobile-toggle + clase open implementado. Validar en dispositivo.

CP-099 Compatibilidad de navegadores

MEDIA**PENDIENTE**

Precondiciones	Chrome, Edge y Firefox actualizados.
Pasos	1. Ejecutar el flujo completo en cada navegador.
Resultado esperado	Comportamiento idéntico.
Resultado obtenido	Pendiente de validación multi-navegador.

CP-100 Zoom 200%

BAJA**PENDIENTE**

Precondiciones	Navegador con zoom 200%.
Pasos	1. Recorrer las pantallas principales.
Resultado esperado	Sin elementos cortados ni superpuestos.
Resultado obtenido	Pendiente de validación.

17. UI/UX — Cambios recientes

Verificación de los últimos cambios de interfaz (logo real, botones amarillos, tarjeta de monedas rotando).

CP-101 Logo real (JPEG) en login, registro y dashboard		MEDIA	APROBADO
Precondiciones	Build desplegado con src/assets/logo.jpg.		
Pasos	1. Ver login, registro y la barra superior del dashboard.		
Resultado esperado	El logo imagen reemplaza el texto "divise." y el SVG anterior; se ve redondeado con borde dorado.		
Resultado obtenido	Implementado y desplegado; el asset se sirve con HTTP 200. Correcto.		

CP-102 Botones de login/registro amarillos (correlación con el tema)		MEDIA	APROBADO
Precondiciones	Páginas de login y registro.		
Pasos	1. Observar el botón primario en ambas pantallas.		
Resultado esperado	Botón amarillo/dorado (#F0B90B) consistente con el resto de la app.		
Resultado obtenido	Se removió el override azul de Auth.css; usa .btn-primary global (amarillo). Correcto.		

CP-103 Tarjeta de mercado rota monedas automáticamente		BAJA	APROBADO
Precondiciones	Login o registro con AuthMarketCard.		
Pasos	1. Observar la tarjeta durante 15-20 segundos.		
Resultado esperado	La moneda cambia cada 3 segundos con animación (Dólar Blue, Oficial, MEP, Euro, BTC, ETH).		
Resultado obtenido	setInterval 3s + key para re-animar. Implementado. Correcto.		

CP-104 Redirección de la raíz "/"		BAJA	APROBADO
Precondiciones	Sesión no iniciada.		
Pasos	1. Abrir la URL raíz.		
Resultado esperado	Redirige al login (vía /dashboard protegido).		
Resultado obtenido	"/" → Navigate a /dashboard → PrivateRoute → /login. Correcto.		

18. Observaciones y Defectos Conocidos

Resumen de hallazgos del análisis de código y ejecución parcial. Priorizar corrección antes de la puesta en producción definitiva.

- F1 (Crítico).** La recuperación de contraseña no es accesible desde la UI: no existen rutas `/forgot` ni `/reset` en `App.jsx`. El enlace "¿Olvidaste tu contraseña?" redirige de vuelta al login. Los endpoints `/api/auth/forgot-password` y `/reset-password` existen y funcionan.
- F2 (Crítico).** Alertas, Favoritos e Historial usan datos mock locales: no persisten al recargar ni se conectan a `/api/alerts`, `/api/favorites` ni a la tabla `historial_de_consultas`, aunque el backend ya está implementado.
- F3 (Alto).** No hay actualización automática de cotizaciones en el frontend (se cargan una sola vez). El backend sincroniza cada 5 min pero el cliente no re-consulta. Falta `polling/websocket`.
- F4 (Alto).** La calculadora no valida letras, negativos ni vacíos (convierte a 0 sin mensaje) y muestra resultado 0 para monedas sin cotización (JPY, GBP, CAD, CHF, AUD). Los botones "Convertir" y "Agregar a favoritos" no tienen acción.
- F5 (Alto).** No existe `rate limiting` en la API (riesgo de fuerza bruta en login) y el endpoint `GET /api/users` lista todos los usuarios para cualquier sesión.
- F6 (Medio).** El login no normaliza el email (mayúsculas/espacios fallan). El backend no valida la longitud mínima de contraseña (solo el frontend).
- F7 (Medio).** Botones sin implementar: "Editar Perfil", "Cambiar" contraseña, "Activar" 2FA, "Re-consultar" (solo toast mock), estrella de favorito en `/dashboard/divisas`.
- F8 (Medio).** El historial no filtra por fechas (solo muestra un toast). El "Recargar favoritos/historial" no consulta datos reales.
- F9 (Bajo).** Noticias depende de `api.rss2json.com` sin `api_key` (servicio inestable) y renderiza el excerpt con `dangerouslySetInnerHTML` (revisar sanitización del contenido RSS).
- F10 (Bajo).** Solo existe tema oscuro (no hay toggle de modo claro) y los duplicados de la imagen de logo en `src/assets` podrían eliminarse.

19. Resumen del Ciclo de Pruebas

ESTADO	CANTIDAD (APROX.)	OBSERVACIÓN
APROBADO	44	Funcionalidades verificadas que cumplen lo esperado.
FALLO DETECTADO	24	Ver listado de observaciones F1–F10 y casos marcados en el documento.
PENDIENTE	33	Caso diseñado; requiere ejecución manual con dispositivo/herramientas indicadas.

N/A	3	Modo oscuro/tema claro (no existe toggle).
-----	---	--

Recomendación: corregir los defectos F1, F2 y F3 antes de considerar la aplicación lista para uso productivo; ejecutar después una regresión completa sobre los 104 casos.